



OBJECT ORIENTED SYSTEMS (OOS) — 2025

QUESTION 2 COMPLETE DETAILED SOLUTION



Introduction

Question 2 of the 2025 Object Oriented Systems examination belongs to CO2 and focuses on some of the most practical and critically important topics in Java programming:

- 🔥 Multithreading
- 🔥 Thread Lifecycle
- 🔥 Synchronization
- 🔥 `wait()` and `sleep()`
- 🔥 `interrupt()` and `interrupted()`
- 🔥 Race Condition
- 🔥 Deadlock
- 🔥 Runnable Interface
- 🔥 File Handling
- 🔥 Scanner and BufferedReader

These concepts are extremely important because modern software systems rarely execute only one task at a time 🚀🕒. Real-world applications such as web servers, operating systems, multiplayer games, Android applications, banking systems, and enterprise backend systems all depend heavily on concurrent execution, synchronization, and efficient resource sharing.

Without proper multithreading mechanisms, applications become slow, CPU utilization decreases, responsiveness suffers, and resource conflicts occur ⚠️. Thus this question is designed to evaluate both conceptual understanding and practical runtime behavior analysis 🧠.



QUESTION 2(a)(i)

? **Distinguish between `sleep()` and `wait()` methods in Java.**

✅ Answer

The methods `sleep()` and `wait()` are both used to pause thread execution temporarily in Java, but they belong to entirely different mechanisms and serve very different purposes 🚀.

Although beginners often confuse these methods because both cause thread suspension, their ownership, synchronization behavior, lock management, and runtime functionality are completely different.



`sleep()` Method

The method `sleep()` belongs to the `Thread` class. It temporarily pauses the currently executing thread for a specified duration. During sleep, the thread remains inactive, but it **DOES NOT release object locks** ⚠️.



Example of `sleep()`

```
class Demo {  
    public static void main(String args[]) throws Exception {  
        System.out.println("Start");  
        Thread.sleep(2000);  
        System.out.println("End");  
    }  
}
```

Output

```
Start  
(2 second pause)  
End
```

💡 **Deep Explanation** The statement `Thread.sleep(2000)` suspends the current thread for 2000 milliseconds. However, if the thread owns any lock, the lock remains occupied. Thus other waiting threads cannot access synchronized resources.

💡 `wait()` Method

The method `wait()` belongs to the `Object` class. It causes the current thread to release the object lock, enter a waiting state, and wait until another thread invokes `notify()` or `notifyAll()`.

Example of `wait()`

```
class Demo {  
    synchronized void test() throws Exception {  
        System.out.println("Waiting...");  
        wait();  
        System.out.println("Resumed");  
    }  
}
```

💡 **Deep Explanation** Unlike `sleep()`, `wait()` releases the monitor lock. Thus other synchronized threads can proceed 🚀.

💡 Major Differences

Feature	<code>sleep()</code>	<code>wait()</code>
Belongs To	<code>Thread</code> class	<code>Object</code> class
Lock Released	No	Yes
Synchronization Required	No	Yes
Purpose	Pause thread	Inter-thread communication

Feature	<code>sleep()</code>	<code>wait()</code>
Wakeup Mechanism	Time completion	<code>notify()</code> / <code>notifyAll()</code>

☀️ JVM Perspective

Internally, `sleep()` interacts with the thread scheduler, while `wait()` interacts with the object monitor system. Thus their runtime mechanisms differ significantly 🧠.

🎯 Conclusion

The `sleep()` method pauses thread execution without releasing locks, whereas `wait()` releases object locks and supports inter-thread communication 🚀.

📄 QUESTION 2(a)(ii)

❓ Distinguish between `interrupt()` and `interrupted()` methods.

✅ Answer

The methods `interrupt()` and `interrupted()` are both related to thread interruption mechanisms in Java, but their purposes and behaviors are fundamentally different 🚀.

☀️ `interrupt()` Method

The method `interrupt()` belongs to the `Thread` class. It is used to signal a thread that interruption has been requested. It does **NOT** forcibly terminate thread execution ⚠️. Instead, the interruption flag becomes set, and certain blocking operations throw an `InterruptedException`.

💻 Example of `interrupt()`

```
class Demo extends Thread {
    public void run() {
        try {
            Thread.sleep(5000);
        } catch (Exception e) {
            System.out.println("Interrupted");
        }
    }

    public static void main(String args[]) {
        Demo d = new Demo();
        d.start();
        d.interrupt();
    }
}
```

💻 Output

Interrupted

☀ interrupted() Method

The method `interrupted()` is static. It checks whether the current thread has the interruption status. It also clears the interruption flag after checking.

💻 Example of `interrupted()`

```
class Main {  
    public static void main(String args[]) {  
        Thread.currentThread().interrupt();  
        System.out.println(Thread.interrupted());  
    }  
}
```

💻 Output

true

☀ Comparison Table

Feature	<code>interrupt()</code>	<code>interrupted()</code>
Type	Instance Method	Static Method
Purpose	Sends interruption request	Checks interruption status
Affects	Target thread	Current thread
Return Type	<code>void</code>	<code>boolean</code>

🎯 Conclusion

The `interrupt()` method requests thread interruption, whereas `interrupted()` checks and clears the interruption status of the current thread 🚀.

📄 QUESTION 2(a)(iii)

? Distinguish between `readLine()` and `nextLine()` methods.

✅ Answer

Both `readLine()` and `nextLine()` are used for reading text input in Java, but they belong to different classes and behave differently 🚀.

☀️ `readLine()`

The method `readLine()` belongs to the `BufferedReader` class. It reads the entire line as a `String`. It is faster because `BufferedReader` uses internal buffering.

💻 Example of `readLine()`

```
import java.io.*;

class Main {
    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String s = br.readLine();
        System.out.println(s);
    }
}
```

☀️ `nextLine()`

The method `nextLine()` belongs to the `Scanner` class. It also reads the entire line but provides easier parsing support.

💻 Example of `nextLine()`

```
import java.util.*;

class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(s);
    }
}
```

☀️ Comparison Table

Feature	<code>readLine()</code>	<code>nextLine()</code>
Class	<code>BufferedReader</code>	<code>Scanner</code>
Speed	Faster	Slower
Parsing Support	No	Yes
Exception Handling	Mandatory (throws <code>IOException</code>)	Not mandatory

🎯 Conclusion

`readLine()` provides faster buffered input whereas `nextLine()` offers easier parsing and user-friendly input handling 🚀.

QUESTION 2(a)(iv)

? **Distinguish between `nextInt()` and `hasNextInt()` methods.**

✓ Answer

The methods `nextInt()` and `hasNextInt()` belong to the `Scanner` class, but they perform completely different operations 🚀.

🌟 `nextInt()`

The method `nextInt()` directly reads integer input from the scanner stream. If the input is invalid, an exception occurs ⚠️.

Example

```
Scanner sc = new Scanner(System.in);
int x = sc.nextInt();
```

🌟 `hasNextInt()`

The method `hasNextInt()` checks whether the next token is an integer. It returns `true` or `false` without consuming the input.

Example

```
Scanner sc = new Scanner(System.in);
if(sc.hasNextInt()) {
    int x = sc.nextInt();
}
```

🌟 Comparison Table

Feature	<code>nextInt()</code>	<code>hasNextInt()</code>
Purpose	Reads integer	Validates integer
Return Type	<code>int</code>	<code>boolean</code>
Exception Risk	Yes	No
Input Consumption	Yes	No

Conclusion

`nextInt()` reads integer values directly whereas `hasNextInt()` safely checks input validity before reading



QUESTION 2(b)

? Fill up the blanks with appropriate phrases. Hence justify the validity of each completed sentence with proper arguments. Provide code snippets where necessary.

QUESTION 2(b)(i)

? "It is better to create threads by _____ than _____."

 Correct Answer

 **"It is better to create threads by implementing Runnable interface than extending Thread class."**

Detailed Explanation

In Java, threads can be created mainly in two ways: by extending the `Thread` class, or by implementing the `Runnable` interface . Although both approaches are valid, implementing `Runnable` is generally considered better because Java supports single inheritance only. If a class already extends another class, it cannot extend `Thread` simultaneously . However, by implementing `Runnable`, the class remains free to inherit another parent class. Thus `Runnable` provides better flexibility, loose coupling, improved design, and cleaner object-oriented architecture.

Example Using Runnable

```
class Demo implements Runnable {
    public void run() {
        System.out.println("Thread Running");
    }
}

class Main {
    public static void main(String args[]) {
        Demo d = new Demo();
        Thread t = new Thread(d);
        t.start();
    }
}
```

Output

```
Thread Running
```

Conclusion

Implementing **Runnable** is preferred over extending **Thread** because it supports better inheritance flexibility and cleaner software design 🚀.

📄 QUESTION 2(b)(ii)

? **"Synchronized _____ ensure more concurrency than synchronized _____ because _____."**

✅ Correct Answer

🔥 **"Synchronized blocks ensure more concurrency than synchronized methods because only critical sections are locked."**

💡 Detailed Explanation

Synchronization is used in Java to protect shared resources, prevent race conditions, and ensure thread safety 🚀. Synchronization can be achieved using synchronized methods or synchronized blocks. A synchronized method locks the entire method. However, a synchronized block locks only a specific portion of code. Thus synchronized blocks provide finer control, reduced waiting time, and better concurrency ⚡.

💻 Example of Synchronized Block

```
class Demo {  
    void display() {  
        System.out.println("Non Critical Section");  
  
        synchronized(this) {  
            System.out.println("Critical Section");  
        }  
    }  
}
```

💡 Deep Conceptual Analysis

In synchronized methods, the whole method becomes locked. Even non-critical operations remain blocked. But synchronized blocks lock only the resource-sensitive section. Thus multiple threads can execute remaining parts simultaneously 🚀.

🎯 Conclusion

Synchronized blocks improve concurrency because only critical sections are locked instead of entire methods.

📄 QUESTION 2(b)(iii)

? **"Race around condition occurs in _____ and is eliminated by _____."**

✅ Correct Answer

🔥 **"Race around condition occurs in multithreading and is eliminated by synchronization."**

🌟 Detailed Explanation

A race condition occurs when multiple threads access shared data simultaneously, and the program output depends upon unpredictable thread execution order ⚠️. This causes inconsistent output, corrupted data, and unpredictable behavior. Synchronization prevents this problem by allowing only one thread at a time to access critical shared resources 🚀.

💻 Example Without Synchronization

```
class Counter {  
    int count = 0;  
    void increment() {  
        count++;  
    }  
}
```

Multiple threads modifying `count` simultaneously may produce incorrect results.

💻 Correct Version

```
class Counter {  
    int count = 0;  
    synchronized void increment() {  
        count++;  
    }  
}
```

🌟 Deep Conceptual Analysis

The keyword `synchronized` uses monitor locking. Only one thread enters the critical section at a time. Thus data inconsistency disappears 🚀.

🎯 Conclusion

Race conditions occur in multithreaded environments and are prevented using synchronization mechanisms.

📄 QUESTION 2(b)(iv)

❓ "Deadlock can be prevented by _____ so that no _____ occurs."

✅ Correct Answer

🔥 "Deadlock can be prevented by proper resource allocation so that no circular waiting occurs."

🌟 Detailed Explanation

Deadlock is a dangerous condition in multithreading where multiple threads wait indefinitely for resources held by one another ⚠️. One of the major causes of deadlock is circular waiting.

Example:

- Thread A waits for a resource held by Thread B.
- Thread B waits for a resource held by Thread A.

Neither thread proceeds. Thus the program hangs permanently 🚀. Proper lock ordering and resource allocation prevent this problem.

Example of Deadlock Prevention

```
class Demo {  
    static final Object lock1 = new Object();  
    static final Object lock2 = new Object();  
  
    void method1() {  
        synchronized(lock1) {  
            synchronized(lock2) {  
                System.out.println("Safe Execution");  
            }  
        }  
    }  
}
```

🌟 Deep Conceptual Analysis

If all threads acquire resources in the same order, a circular dependency never forms. Thus deadlock gets prevented 💬.

🎯 Conclusion

Deadlock prevention requires proper resource allocation strategies so that circular waiting never occurs.

QUESTION 2(b)(v)

? **"Inter thread communication is achieved by ____."**

✅ **Correct Answer**

🔥 **"Inter thread communication is achieved by wait(), notify(), and notifyAll() methods."**

🌟 Detailed Explanation

Inter-thread communication is a mechanism through which multiple threads coordinate execution, share information, and synchronize activities 🚀. Java provides three important methods from the `Object` class for inter-thread communication:

- `wait()`

- `notify()`
- `notifyAll()`

These methods operate using object monitor locks.

- `wait()`: Causes the current thread to release the lock and enter a waiting state.
- `notify()`: Wakes up one waiting thread.
- `notifyAll()`: Wakes up all waiting threads.

Example

```
class Demo {  
    synchronized void test() throws Exception {  
        wait();  
        System.out.println("Thread Resumed");  
    }  
  
    synchronized void resumeThread() {  
        notify();  
    }  
}
```

Deep Conceptual Analysis

These methods enable producer-consumer communication, coordinated execution, resource sharing, and efficient synchronization .

Conclusion

Inter-thread communication in Java is achieved using `wait()`, `notify()`, and `notifyAll()` methods.

QUESTION 2(c)

? Write a Java class having a method that opens a text file a.txt in read mode, select all the palindrome words and write them into another file b.txt. Hence count the number of such words in the file. OR ? Write a Java class having a method that opens a text file in read mode, select all the words containing at most one vowel and write them into another file. Hence count the number of such words in the file. Here the file names should be taken as input from the user.

 Answer (Palindrome Word Program)

Detailed Explanation

A palindrome word is a word that remains identical even after reversing its characters  (e.g., `madam`, `level`, `civic`, `radar`).

The program must:

1. Open file `a.txt` in read mode.

2. Read all words from the file.
3. Check whether each word is a palindrome.
4. Write palindrome words into `b.txt`.
5. Count total palindrome words.

This question mainly tests Java File Handling, `BufferedReader`, `FileWriter`, String processing, and logical programming 🗨️.

Java Program

```
import java.io.*;

class PalindromeFile {
    static boolean isPalindrome(String word) {
        String rev = "";
        for(int i = word.length() - 1; i >= 0; i--) {
            rev += word.charAt(i);
        }
        return word.equalsIgnoreCase(rev);
    }

    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new FileReader("a.txt"));
        BufferedWriter bw = new BufferedWriter(new FileWriter("b.txt"));

        String line;
        int count = 0;

        while((line = br.readLine()) != null) {
            String words[] = line.split("\\s+");

            for(String w : words) {
                if(isPalindrome(w)) {
                    bw.write(w);
                    bw.newLine();
                    count++;
                }
            }

            br.close();
            bw.close();

            System.out.println("Palindrome Words = " + count);
        }
    }
}
```

Sample Input (`a.txt`)

```
madam apple level java civic radar
```

Output File (b.txt)

```
madam  
level  
civic  
radar
```

Console Output

```
Palindrome Words = 4
```

Deep Conceptual Analysis

- `readLine()` reads the file line-by-line.
- `split("\\s+")` separates words using spaces.
- The palindrome check reverses the string manually and compares it with the original string.
- Matching words are written into `b.txt` using `BufferedWriter`.

Why `BufferedReader` and `BufferedWriter` are Used: Buffered streams improve input-output performance, memory efficiency, and file processing speed 🚀.

Conclusion

The program successfully reads a text file, detects palindrome words, writes them into another file, and counts their occurrences using Java file handling and string processing mechanisms 🚀.

Answer (OR PART - One Vowel Words Program)

Detailed Explanation

This program is based on Java File Handling, String Processing, Character Analysis, and User Input Handling 🚀☕.

The objective of the program is:

1. Open a text file in read mode.
2. Read all words from the file.
3. Check how many vowels each word contains.
4. Select only those words having zero vowels or exactly one vowel.
5. Write such words into another output file.
6. Count total number of such words.

This question is extremely important because it combines file processing, loops, conditional logic, methods, and string manipulation 🧠.

🌟 **Understanding “At Most One Vowel”** The phrase 🔥 **“At most one vowel”** means the word may contain zero vowels or one vowel only.

- sky (0 vowels) ✓
- cat (1 vowel) ✓
- apple (2 vowels) ✗

🌟 Approach of the Program

1. Take input and output file names from the user.
2. Open input file using `BufferedReader`.
3. Read file line-by-line.
4. Split each line into words.
5. Count vowels inside every word.
6. If vowel count ≤ 1 , write the word into the output file.
7. Maintain total count and display final count.

💻 Java Program

```
import java.io.*;
import java.util.*;

class OneVowelWords {
    static int countVowels(String word) {
        int count = 0;
        word = word.toLowerCase();

        for(int i = 0; i < word.length(); i++) {
            char ch = word.charAt(i);
            if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u') {
                count++;
            }
        }
        return count;
    }

    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Input File Name: ");
        String inputFile = sc.nextLine();

        System.out.print("Enter Output File Name: ");
        String outputFile = sc.nextLine();

        BufferedReader br = new BufferedReader(new FileReader(inputFile));
        BufferedWriter bw = new BufferedWriter(new FileWriter(outputFile));
```

```
String line;
int total = 0;

while((line = br.readLine()) != null) {
    String words[] = line.split("\\s+");

    for(String w : words) {
        int vowels = countVowels(w);
        if(vowels <= 1) {
            bw.write(w);
            bw.newLine();
            total++;
        }
    }

    br.close();
    bw.close();

    System.out.println("Words containing at most one vowel = " + total);
}
```

Sample Input File (a.txt)

sky fly cat apple dry book cry jump

Output File (b.txt)

sky
fly
cat
dry
cry
jump

Console Output

Words containing at most one vowel = 6

Step-by-Step Program Analysis

- The method `countVowels()` counts total vowels inside a word.
- The condition `if(vowels <= 1)` ensures only words containing zero or one vowel are selected.
- The method `split("\\s+")` splits text into individual words.

- `BufferedReader` reads the file efficiently line-by-line.
- `BufferedWriter` writes selected words into the output file efficiently 🚀.

💡 Why Buffered Streams are Preferred

Buffered streams improve execution speed, memory efficiency, and file handling performance. Instead of accessing the disk repeatedly, data is temporarily buffered in memory. Thus file operations become much faster ⚡.

💡 Real-world Applications

Programs like this are useful in text analytics, spell checking, natural language processing, search engines, document filtering, and AI preprocessing systems 🧠.

🎯 Conclusion

The program successfully reads a text file, identifies words containing at most one vowel, writes them into another file, and counts their occurrences efficiently using Java file handling and string processing mechanisms 🚀🧠. This program demonstrates practical usage of file handling, string manipulation, conditional logic, and efficient buffered I/O systems in Java 🧠💡.